

Programming Language Design

Module 3: Intro to Haskell

Slides © Grant Williams, [CC BY-SA 4.0](#).

This is an open educational resource.

Feel free to submit fixes, improvements, and new material [here](#).

Last Module

We learned about Lambda Calculus.

Lambda Calculus is a model for computation that treats every operation as either creating a function or calling it.

Lambda calculus seems weird, but hopefully it became clear that you could do anything in it.

Now it's time to put it to use...

This module

We're going to start learning Haskell.

Haskell is a pure functional programming language.

It has several weird features that you have never seen before.

So, without further ado...

History of Haskell

Haskell was designed by a committee to specifically be a Lazy functional programming language. (We'll explain what this means later).

This has a specific technical meaning. It doesn't mean that it's chill. It's actually way weirder than that.

There are several Haskell compilers, but the main one is [GHC](#), the Glasgow Haskell Compiler, named after the University of Glasgow where it was developed.

Basic programming language facts

Haskell is (usually) a compiled language, meaning that we run the compiler on Haskell source code to generate an executable that can be run later.

Haskell is statically typed, meaning that the types of variables and expressions is known at compile time ("statically" means at compile time, "dynamically" means at run time).

Unlike C or other languages you may be familiar with, Haskell is implicitly typed. It is good at figuring out the types of expressions without you having to write them. Writing them is usually optional (but still a good idea for public functions).

Haskell is *not* object-oriented. The class keyword is unrelated to Classes in an OO language. Instead, we organize our programs around functions, instead of classes.

Functional programming

What is functional programming?

Fundamentally, functional programming means that your computer program is a function. The functional programming paradigm treats all programs as functions.

But wait, does that mean C is a functional language? After all, the program starts with the `main` function!

No, because `main` (usually) is not actually a function. At least, a mathematician would not consider it to be a function.

So what is a *real* function, then?

Real functions are *pure*

Functions that we care about in this class are *pure* functions.

A pure function has two properties:

1. Referential transparency. That is, every time the function is called with a particular argument, it should give the same result. The functions output should depend only on its input (and maybe constants).
2. Lack of side effects. That is, calling the function should not change anything elsewhere. This may sound extreme: it is! Variables aren't allowed to change.

Let's examine these two properties.

Referential transparency

In math, suppose I define a function: $f(x) = 2x$

So $f(2) = 4$, and $f(7) = 14$

Those are not temporary values, they are definitional. $f(7)$ is 14 because 14 is 2×7 .

This would make no sense:

$f(7) = 14$; $f(7) = 15$; $f(7) = 16$

A mathematician looking at that would not accept it. "Which is it? Is $f(7)$ 14, 15, or 16?"

The term "referential transparency" can be hard to remember, so let's see where it comes from...

Referential transparency (2)

Discussion from an [interesting stackoverflow post](#) (top response)

The term comes from analytic philosophy. The idea was to determine which kind of verbs could simply have their "referrents" (the things they were about) replaced transparently.

For example, in the phrase "Football is watched by millions of Americans every year", "Football" can be replaced by "America's most popular sport" without altering the meaning of the sentence.

However, in the phrase "I watched the football game", we cannot say "I watched the America's most popular sport game."

So "is watched by" is referentially transparent here, but "I watched the" is not.

Referential transparency (3)

This idea was adapted to functional programming languages by treating arguments to functions as "referents".

A referentially transparent function is one in which all the references to its arguments can be replaced by the argument.

So that function from earlier, $f(x) = 2x$ is referentially transparent, because $f(7)$ can be rewritten as $f(7) = 2(7)$ without getting something wrong.

Can anyone think of a function that is *not* referentially transparent.

Referential transparency (4)

`input()` in Python is one example of a non-referentially transparent function.

Sometimes, `input()` returns "Hello". Other times, it returns "Bob". It returns whatever string is in standard input at that time.

But in Haskell, this would not be possible. It has to always return the same thing given the same arguments.

And since `input` has no arguments, that means it has to always return the same thing. It would have to be a constant.

Another example: in C, `scanf` returns the number of format specifiers it matched. That's going to depend on what's in standard input.

No side effects

The other aspect of pure functions is that they have no *side effects*.

A side effect is any kind of IO or variable change.

So this means, a pure function never modifies a variable. It also never writes to a file, inserts into a database, or sends an internet packet.

Haskell *only* has pure functions.

So how does it do any of those things? Can you just not write database programs in Haskell?

No, you can, you just have to change how you think about it. Don't worry, we'll see some examples of writing useful software in Haskell. There are [games](#), [office software](#), and even a [Linux window manager](#) written in Haskell.

Questions?

Lazy Evaluation

Almost every programming language you have ever used, is *eagerly* or *strictly* (both mean the same thing) evaluated.

What does that mean?

It means that expressions are evaluated in the order they are arrived at.

That seems so straightforward that it's impossible to imagine anything different, so here's a concrete example.

Eager evaluation in C

C is a (mostly) eagerly evaluated language.

Consider this C code:

```
int main() {}  
    int x = 8 + 8;  
    int y = 4 + 4;  
    printf("x is %d\n", x);  
    return 0;  
}
```

What does it do?

It first computes $8 + 8$, and stores the result in a variable called `x`. It then computes $4 + 4$ and stores the result in an unused variable `y`. It then prints out `x`. Finally, it exits with a code of 0.

How else could it possibly work?

Here is a similar looking bit of Haskell code:

```
main = do
  let x = 8 + 8
  let y = 4 + 4
  print x
```

What does it do?

It computes $8 + 8$ and binds the result under the name `x`. Then it prints `x`.

It *does not* compute `y`.

Why? Because it doesn't matter. You never print it, so it is never used.

How does it know?

Fundamentally, Haskell evaluates expressions from the "other direction".

Whatever actions are present in main need to happen to matter what. So it sees `print` `x` and realize that it needs to compute `x` at that point.

Then it works backward and actually computes it, which means `8 + 8`.

Nothing ever forces it to compute `y`, so it never does.

There was an old Haskell benchmark that was supposed to compute digits of pi or something. The benchmark creator forgot to print, so briefly Haskell was the fastest language in the world, because the program ran in 0 seconds.

Why?

Lazy evaluation isn't actually common, even in functional programming languages.

Apparently, it was chosen as a way to *force* Haskell to be functionally pure.

If you can't rely on a particular order of computation, you can't have side effects.

It can be faster under certain circumstances. If there's some variable that is expensive to compute and only occasionally necessary, you can save the cost if it isn't used.

It also means you can easily have infinite data structures without iterators. Nothing stops you from creating a list of all integers, because each element is created as it is read in the list, and not all at once.

Why not?

If lazy evaluation is so great, why isn't it more common?

Unfortunately, it makes memory usage very unpredictable.

When haskell sees an expression, it creates a special value that kind of encodes the computation without running it (called a "thunk").

If the expression is a large or infinite list, as you pull data from it, you end up allocating a lot of memory all at once.

It's possible to disable this behavior, but the fact that it's the default means that software that needs to be easily speed/memory audited tends to not be written in Haskell.

Questions?

So what about "print"?

But now, let me address a question you may have: if functions are all pure and have no side-effects, what does this mean?

```
main = putStrLn "Hello, World"
```

It's clearly printing! It's doing something! How can you say that all functions in Haskell are pure?

I didn't lie!

That function has no side effects.

"But it prints!" No it doesn't!

That function actually does not print anything.

Consider the strange syntax. Notice that we write: `main = ...` and not `main { ... }` or `main: ...`. Why is it an equals sign?

Because it is a mathematical definition. We are saying that `main` is a constant that contains the value `putStrLn "Hello, World"`.

But `putStrLn` is a function, not a value, right?

It's a value

Let's write the type above the program:

```
main :: IO ()  
main = putStrLn "Hello, World"
```

You've already seen types in your readings so far. We write them with `::`. This is saying that `main` is something called an `IO`, and then there's this `()` next to it.

`IO` is a kind of *monad*, a concept we will learn in detail much later in the course.

However, we can think of an `IO` as a kind of program. An `IO` is a program that performs input and output when it is run.

When does it run?

When is it run? Not by us! We are returning it from `main`. The Haskell runtime is going to actually run it.

So *in Haskell*, every function is pure. But the function can return a program, and a different system will run that program. It can have side effects.

In fact, it probably *should* have side effects. Otherwise you could never print anything!

The `()` after `IO` is the return type of the program. In this case, the program doesn't return anything. `()` is pronounced "unit", and it represents a type of value that is irrelevant or for which there is only one possibility (like "nothing"). It's kind of like null, but it's not only a value, but also a type.

`IO Int` would be an `IO` program that returns an `Int`. But main must be `IO ()`.

Re-iterating that

In Haskell, *main is not a function*.

It is a constant. It is a program that never changes. Every time the runtime runs your program, it will reach inside and pull the program out of the `main` variable, and then run it.

The program is allowed to have side effects. It can print and read and open files.

However, `main` will always return the same program. Even if the program has side-effects, `main` is not running the program, only returning it.

This is the mindset shift that we need to do in order to really *get* functional programming. Nothing changes; everything is a map from input to output.

The lazy evaluation strategy

Inside your program, Haskell doesn't actually do anything.

When you return a value as `main`, the runtime starts evaluating the program you returned. This is when it starts evaluating expressions, and lazy evaluation happens.

If `main` doesn't use a value as part of IO it never gets evaluated.

I'll prove it:

```
main :: IO ()
main = do
    let x = putStrLn "hi"
    return ()
```

This doesn't print anything. `putStrLn` is not evaluated.

How does outside code get called?

So how can someone call a C library from Haskell?

There is a way to do it. There are ways to call native programs.

There's also a function called `unsafePerformIO` that allows you to use a value computed from an IO action inside a pure function.

But it still doesn't actually run until the runtime's lazy evaluator makes it.

Questions?

Big changes from imperative programming

Okay, so functional programming is weird. But if I kind of look at that earlier Haskell code side by side with the C code, it kind of looks the same! What's the big difference?

Well, for one thing, you can't do this:

```
x = 20  
x = 10
```

That's not valid. `x` can only be one thing.

Big changes (2)

You *can* do this:

```
do  
x <- return 20  
x <- return 10  
print x
```

But it doesn't actually change `x`, it creates a different value in a different scope. (we'll talk about the weird arrow later).

So that means we can't have loops!

No loops

In C, a loop is a structure that executes a series of statements as long as a condition is true. Then it breaks when the condition fails.

In Haskell, this would never happen, because the condition would either be true or false, and that would never change. Haskell does not have C-style loops.

(There are alternatives, including things like for-each, but they involve those monad things, so let's not worry about it for now)

So what do we do if we can't loop? Surely there has to be some kind of way? [What's the alternative?]

Recursion instead

Instead we use recursion.

Let's write a simple loop over a list to compute the sum.
This already exists, but it's a simple exercise.

Here's how we'd do it in C

```
void sum(int* arr, size_t n) {  
    int sum = 0;  
    for(size_t i = 0; i < n; i++)  
        sum += arr[i];  
    return sum;  
}
```

But in Haskell...

Lists

```
sum'' :: [Int] -> Int
sum'' list =
    if list == [] then 0
    else head list + sum'' (tail list)
```

This is a recursive function. It takes a list of ints and returns an int.

It first checks if its argument is empty. If it is, the sum is zero.

Otherwise, it returns the head of the list plus the sum of the tail.

Take a second to convince yourself it works, and consider that we just "looped" without modifying anything.

Running it

Since you've been working the book exercises, I don't need to put this here, but in the incredibly unlikely scenario you've been holding off on installing Haskell...

Download [GHCup](#) to install GHC, the compiler.

In a new console window, you can write `ghci` to enter interactive mode, in which you type one line at a time, or `ghc <filename>.hs` to compile it to `<filename>.exe` on Windows or just a binary `<filename>` on *nix.

Be sure to do the knowledge checks.

Haskell Basics

Let's understand that code better and review the basics of Haskell that you've already seen from your reading.

First, Haskell is a statically-typed language. Every expression has a well-defined type that is known at compile time.

A Haskell source file is a list of definitions which are largely either functions or constants (ignoring modules for now). Each definition has an equals sign: `=`.

Each definition can have a type explicitly listed, or Haskell can try to infer it. Usually we like to list them, especially for functions, for documentation. Types can be hard to parse by eye.

Expressions vs. statements

A definition looks like this:

```
name :: type  
name arg1 arg2 ... = expression
```

An expression is something that has a value.

A statement is a command that changes state.

Haskell *does not* have statements. It does not permit state to change (within the program). There is no *if statement*. It is an *if expression*.

Every definition, including `main`, has an expression as its right-hand side.

Names

Variables in Haskell aren't really variables, because they can't change.

But we call them that anyway.

Anyway, they can be named the same as C variables: no number up front, but any combination of lowercase, uppercase, and numeric chars.

Haskell usually uses camelCase rather than snake_case, but you can use either.

However, unlike C, you can put a ``` ("prime") anywhere but the first character.

So `hello'` and `world' '` and technically `w' 'orld` (but don't do that) are valid names.

Basic Types

Haskell has a number of basic types that should be pretty familiar:

- Int: a fixed-width int that is at least 29 bits wide.
- Bool: a boolean value (True or False)
- Float: an IEEE-754 32-bit float
- Double: an IEEE-754 64-bit float
- Char: a unicode codepoint*
- Integer: a bigint that can store integers of any length that fit in memory.
- (): the "unit" type. Both a type and a value. Used when no meaningful value is necessary.
- most common printed characters have exactly one codepoint unless you're doing Zalgo text or something. Can be more than one codepoint for complicated characters or visual characters.

Compound Types

Haskell also has an equivalent to structs.

They are a little weirder, so we'll be saving that for next module.

However, some important compound types:

- `[Type]`: a list of types. E.g., `[Int]` is a list of Ints. `[Float]` is a list of float.
- `String` is just a renaming of `[Char]`. It's a list of Chars.
- `Ratio` (requires `import Data.Ratio`). A rational number, *not a float*, actually stores the fraction as numerator/denominator as two Integers, so e.g., `3/10` can be represented exactly in binary.
- `IO Type`: a program that, when it is eventually run (by someone else), it will result in "Type". E.g., `IO Int` is a program that will do some I/O and then return an Int.
- `(TypeA, TypeB, ...)`: a tuple. `(Int, Float)` is a pair of an int and a float.

The most important type

But the most important type of all is the function. It's a functional language, after all.

Function types are written with an arrow: `->`

Example: `Int -> Int` is the type of a function that takes an `Int` and returns an `Int`

`Float -> String` is the type of a function that takes a `Float` and returns a `String`.

[What is the type of a function that takes *two* ints and returns an int?]

Functions are curried

```
Int -> Int -> Int
```

Functions are curried by default. This is equivalent to `Int -> (Int -> Int)`.

Consider this function:

```
add :: Int -> Int -> Int  
add a b = a + b
```

If you leave the type off, Haskell will try to infer it.

What type will it infer of this?

```
add 7
```

Currying and partial application

We filled in the first int in an `Int -> (Int -> Int)`

So the result is an `Int -> Int`.

Specifically, it's a function that adds 7 to things.

We can even partially apply the addition operation itself. `(+) :: Int -> Int -> Int`

`(+7)` is a function that adds 7 to things, as is `(7+)`.

And we can write this:

```
add7 :: Int -> Int
add7 = (7+)
```

`add7` has type `Int -> Int`, and so does `(7+)`, so this type checks successfully.

Types and values are different!

If I ask for the type of a function, like `(+)`, the answer is `Int -> Int -> Int`

If I ask for a function itself that adds two values together, you can say:

```
whatever a b = a + b
```

or

```
whatever = (+)
```

or

```
whatever = \a -> \b -> a + b -- these are lambdas btw
```

In Haskell we write `\a -> expr` instead of `$\lambda a.$  expr`, but it's the same thing.

Types and values are different! (2)

A datatype represents a set of values. For example, `Int` is the set of all integers of a certain bit width.

There is another kind of type called a *type class* (not the same thing as an object-oriented class) that we'll talk about later.

A value represents an element of a datatype.

So `Int` is a type, and `7` is a value.

You will never see:

```
someFunction :: 7 -> Int
```

Because `7` is not a type, so you can't write a function that takes the type "`7`".

The distinction

That doesn't mean you can't write a function that only is defined on seven:

```
onlyTakes7 a =  
  if a == 7 then True  
  else error "Ahh!!! Only 7s!!!"
```

But the type of that function is something like `Int -> Bool`, not `7 -> Bool`.

Likewise, we can't have a function return a type:

```
returnsInt = Int
```

You can define type aliases, which is kind of similar, but functions take and return values, and types describe other types. They are two separate categories.

Are they entirely separate? Dependent typing.

In some programming languages, like [this one](#), types and values aren't entirely different.

You could write a function that takes 7. Well, it would take an `Int`, and then a proof that the `Int` is exactly 7.

You could also write a function that takes a list of 7 values.

The type can depend on values, so this is called *dependent typing*.

But Haskell is not a dependently-typed language (unless extensions are used). Types and values are different, and it's very important to understand the difference.

Knowledge Check 1

1. Define a function, named `foo` that takes a Float named `x` and doubles it.
2. What is a type that function could be?
3. Define a function with the type `Float -> Float -> Float -> Float`
4. Define a function that takes an Integer and then ignores it and returns `7`. Give it a type as well.

Knowledge Check 1 answers

1. `foo x = 2.0 * x` or `foo = (2.0 *)`
2. `Float -> Float` or `Double -> Double` are good answers.
3. `evalLinear a b x = a * x + b` is one answer.
- 4.

```
justBe7 :: Integer -> Integer
justBe7 x = 7
-- or
justBe7 :: Integer -> Integer
justBe7 = const 7
```

`const` is a function that takes a value, and then returns a function that throws its argument away and returns the value you gave to `const`.

Questions?

More about const

It's sometimes possible to pretend functional programming languages are normal.

But `const` illustrates how they're not.

`const` is a function that returns a function. And the function it returns just returns whatever value you gave `const`.

It's kind of a good way to determine if you've mastered lambda calculus. [Try writing it?]

Const

A few options, assuming the type is `Int`:

```
const'  :: Int -> Int -> Int
const' x = \ _ -> x
-- or
const'' :: Int -> Int -> Int
const'' x _ = x
```

The top one returns a lambda that returns `x`.

The bottom one actually does too, because of currying.

In general, if there's a variable we don't care about, we name it `_`. It's a valid name, but it's special because many variables can be named `_` without conflicting.

but what if we don't know the type?

Type parameters

Sometimes we want a function that works with any type, as long as its consistent.

For example `x -> x`, fill in `x`, but make sure the return is the same type.

That's one thing we can do:

```
const '' :: x -> x -> x
const '' x _ = x
```

Every defined type is required to have an uppercase name. If a type name is lowercase, it is a type variable. Also called a type parameter.

Here, the purpose is to make sure that the two values have the same type.

But wait, we don't ever use the second parameter. Why does it need to be the same?
Could we do something else?

The real type of `const`

The actual type is this:

```
const :: a -> b -> a
const x _ = x
```

Notice the `b`. The function that is returned can take anything, it doesn't have to be the same type that is returned.

This is permitted, for example:

```
alwaysReturnHello :: Int -> String
alwaysReturnHello = const "Hello"

-- somewhere else:
print (alwaysReturnHello 29) -- prints "Hello"
```

Parametric type

A type that has at least one type parameter is called a parametric type.

You've already seen these, in Java or C++, an `ArrayList<Integer>` or a `vector<int>` are both parametric types.

You need to know what the list is storing to create it.

In Haskell, there isn't any special syntax to define parameteric types. No angle brackets. You just make the type parameter lowercase.

So instead of `List<T>`, we write `[t]`

Parametric polymorphism

We'll talk more about polymorphism in future modules, but Haskell relies more on parametric polymorphism than Java or C++ do.

Polymorphism is just the idea that a function can do something different based on the types of its arguments.

For example, printing a list is different than printing an integer.

Parametric polymorphism is when the parameters to a function can be different types, and those types are resolved at compile time.

There's also sub-type polymorphism, where we inherit from something to change its behavior. We'll see this later.

Knowledge check 2

1. Define a function that takes three arguments of any type and returns something of the same type as the first argument.
2. Define a function of that type.
3. Define a function that's like `const` , but the function it returns takes two values of any type (instead of one) before ignoring them and returning the argument to `const` (also of any type).
4. What is the type of that function?

Knowledge check 2 answers

1. `x -> y -> z -> x`
2. `pickTheFirstOne a _ _ = a`
3. same as answer 2
4. same as answer 1

Questions?

Back to functions

In functional programming, functions are values just like any other. Just like ints.

What can we do with ints? We can add them, multiply them, all kinds of stuff.

What can we do with functions?

Back to functions (2)

Well, one thing is we can call them. Easy.

We can also define new ones.

But how can we combine two functions? We can add two ints together. Are functions really like that?

[what do you think? Maybe from one of the readings?]

Yes: composition

We can absolutely combine functions.

In Haskell, it uses the 'dot' operator.

`f . g` is "f composed with g".

But what does that mean?

Mathematically

In math, we write $(f \circ g)(x)$ to mean $f(g(x))$

It means "first do g , then do x "

But why bother? For the math you've been introduced to so far, it's not all that important.

But the benefit is, it gives us a way to create new functions in very little code.

Some examples

All of these pairs of functions are equivalent:

```
something x = 2*x + 7  
something' = (+7) . (2*)
```

```
doubleAndPrint x = print (2 * x)  
doubleAndPrint' = print . (2*)
```

```
plus2 x = x + 2  
plus2' = (+2)  
plus2'' = (+1) . (+1)
```

Why?

We'll see how this can be convenient later, but it enables us to think of functions as "pipelines" from one value to another.

With a function like `stuff x = print (2*x + 1)`, notice that your attention has to bounce around. First the `x` gets multiplied by `2`, then we add `1` to it, but then we move back to the left to `print` it.

Instead, `stuff' = print . (+1) . (2*x)` describes what happens as an exact pipeline. First we double, then we add 1, then we print. Done.

This "pipeline" style of programming is central to functional programming. We don't allow variables to change, so the "pipes" that feed one value to another are more important.

Point-free programming

Notice that when we do this, the variable goes away on the left-hand side:

```
stuff x = print (2*x + 1) -- versus  
stuff' = print . (+1) . (2*)
```

The reason is that the composition operators create a function that takes a value, so we don't need a variable.

This style of coding is called "point free". The point in question is the parameter*.

Ironically there are a lot of points in the sense of `.`.

It's an optional coding style that is culturally popular in Haskell. It can be intimidating at first, but you'll get better at reading it.

* Calling parameters "points" comes from homotopic geometry or something.

Knowledge Check 3

1. What is the type of `(*2.5) . (*2.5) . (*2.5)` ?
2. Rewrite that function but not point-free.

Knowledge Check 3 answers

1. `Float -> Float`

2. `f x = 2.5 * 2.5 * 2.5 * x` or `f x = 15.625 * x`

Let, where, and guards

So far, all of our variables have been definitions. At the top level, we've said:

```
someName = some expression
```

This is fine, but we don't want all of our names to be global. Sometimes we just want something named "foo" or "x" and we don't want Haskell to complain that we're redefining this important term.

This is where let-expressions come in.

Let

The `let` keyword "lets" you define one or more temporary constants or functions that are only used in one expression.

For example:

```
add3 =  
  let add1 = (+1)  
      add2 = (+2)  
  in  add1 . add2
```

Here, we're basically saying "first define `add1` and `add2`, and then replace the values in this expression here. The result is `add3`."

Haskell is whitespace sensitive, but it's more flexible than python. Here, the definitions in the `let` need to have the same indentation.

Let-in is an expression

Note: this is not a statement! It is an expression. We can use "let" anywhere an expression is allowed.

```
print (let x = 2 in x * x) will print 4
```

Inside of a `do` block, there is no `in`, the definition just continues throughout the block. We haven't really talked about `do` blocks in detail yet, so we won't say too much more about this now.

So with `let`, we can just simplify complicated expressions into a single variable or define little temporary functions. Both can improve readability.

Where

Sometimes it's nice to define the functions *after* they are needed. It can keep function bodies clean.

For this, there is the `where` keyword:

```
add3andDouble :: Int -> Int
add3andDouble = double . add1 . add2
  where
    double :: Int -> Int
    add1   :: Int -> Int
    add2   :: Int -> Int
    double = (2*)
    add1 x = 1 + x
    add2 = add1 . add1
```

Where (2)

The `where` keyword needs to be more indented than the function it is describing, which is why I gave it that one space.

One major benefit of `where` over `let` is that it lets you define types for the functions.

The types don't have to all be at the top of the `where` block either, I just did that to show you that type declarations don't need to be directly adjacent to the function they are about.

`where` is not an expression though, you can't just put it anywhere. It has to go beneath definitions.

Which should you use? I like to use `let` for small temporaries, and `where` when the temporary functions have complicated types.

Questions?

Important extra features

There are a few Haskell features that are important, but that deeper coverage of which requires some more time.

We'll learn more about these soon, but for now, here's what you need to know.

pattern matching

You can define a function on specific values, and Haskell will pick the definition that matches.

```
f :: Int -> Int
f 0 = 0
f 1 = 1
f x = x + 1
```

In this case, if I write `f 7`, you get `8`, but if I write `f 0` you get `0`.

When you write `f 7`, haskell checks "is it `0`? no, okay is it `1`?" in order from top to bottom. So order matters.

Tuples

What if we want a function to return two values?

Well, we can. Using a tuple:

```
doubleAndAdd3 = Int -> (Int, Int)
doubleAndAdd3 x = (2*x, x + 3)
```

Here, `(Int, Int)` is the type "a pair of ints".

But how do we get values back out?

Tuples (2)

There are two functions, `fst` and `snd` :

```
fst (1, 2) == 1  
snd (1, 2) == 2
```

It feels weird to use functions to unpack a tuple. What if we want a triple? Do we need another set of functions?

```
-- these don't exist  
fstOf3 (1, 2, 3) == 1  
sndOf3 (1, 2, 3) == 2  
thdOf3 (1, 2, 3) == 3
```

Pattern matching with tuples

In fact, we just use pattern matching again.

In Haskell (and many functional languages), **pattern matching is the main way that we get data out of compound data types.**

That is, there's no "dot notation" here. And we don't use array notation for tuples like we do in Python.

Instead, we use definitions to break them apart...

Pattern matching with tuples

One way to use pattern matching to get data out is to define a function on the patterns. This is how `fst` and `snd` are defined:

```
fst :: (a, b) -> a
fst (x, _) = x

snd :: (a, b) -> b
snd (_, y) = y
```

Here, when you call `fst (7, 8)`, Haskell sees that `fst` is defined on a tuple. It will bind `7` to `x`, and `8` to `_` (and remember that `_` means "we don't care"). Then it returns `x`.

Pattern matching with let (2)

Another way to use pattern matching is to use `let`.

Suppose I want to get both the first and second values out of a pair. Suppose I have a pair, `nameAndEmployeeId = ("Bob", "12345")` I could do this (rather inefficiently):

```
-- this is not a great way to do it
employeeData :: String
employeeData =
    let name = fst nameAndEmployeeId -- name == "Bob"
        employeeId = snd nameAndEmployeeId -- employeeId == "12345"
    in "Employee: " ++ name ++ "; Id: " ++ employeeId
```

`++` is the operator for string concatenation.

But this is inefficient. We're having to use two function calls just to get some data out of a tuple. Is there a better way?

Pattern matching with let (3)

Instead of doing that, try this:

```
employeeData :: String
employeeData =
    let (name, id) = nameAndEmployeeId -- we do need the parentheses
    in "Employee: " ++ name ++ "; Id: " ++ employeeId
```

Here, we use pattern matching. When Haskell sees `(x, y)` on the left of an equals, it looks to the right side and unpacks the first element into `x`, and the second into `y`.

This also applies to triples, which brings me to the knowledge check...

Knowledge check 4

1. What should the type of `sndOf3` be?
2. Write an implementation of `sndOf3`.
(Note: implementing a function means defining it)
3. Write both the type and implementation of a function that takes a 4-tuple where the first and last elements are floats, but the middle elements can be anything. The function should return the sum of the first and last element.
4. Is `sumFstAndLst (4, "blip", [1,2,3], 5)` correctly-typed? Haskell will interpret `4` as `4.0`, but what about the string and list?

Knowledge check 4 answers

1. `(a, b, c) -> b`

2. `sndOf3 (_, b, _) = b`

3.

```
sumFstAndLst :: (Float, a, b, Float) -> Float
sumFstAndLst (a, _, _, b) = a + b
```

Note for that last one: values are allowed to have the same name as type parameters.

4. Yes. The middle two shouldn't matter if you wrote the right type.

Questions?

Case

Sometimes we want to do pattern matching outside of a function, but with more than one kind of pattern.

For example, `let` is useful if we want to destructure a tuple: `let (x, y) = pair in ...`

But what if we want to do one thing if `x` is exactly 2?

There's another destructuring expression called `case`.

```
let result =  
  case pair of  
    (2, y) -> y  
    (x, y) -> x
```

This makes `result == y` if `fst pair == 2`, and `snd pair` otherwise.

Guards

Sometimes we want to define a function or expression with more complicated rules than just specifying all the patterns.

Like maybe it does one thing for even values and a different thing for odd values. Or maybe it gives a different result for values greater than 10...

We can do that like this:

```
maxEsrbRatingForAge :: Int -> String
maxEsrbRatingForAge age
  | age <= 3 = "EC" -- the indentation is significant
  | age <= 12 = "E"
  | age < 17 = "T"
  | otherwise = "M"
```

Guards (2)

This is equivalent to using ifs, but nicer:

```
maxRating age =  
  if age <= 3 then "EC"  
  else if age <= 12 then "E"  
  else if age < 17 then "T"  
  else "M"
```

Guards have a nicer syntax, but they can only be used when defining functions.

Read and show

How do we cast a value to a string?

In Haskell, there isn't really a special "cast" operator. We don't do like `(String)20` or something. Instead, we call a function.

The function for converting things into a string is `show`.

For example, `show 20 == "20"`.

If you look at the type of `show`, you get something a little weird:

```
show :: Show a => a -> String
```


Read and show (2)

What is the capitalized `Show` ? That's called a typeclass.

Despite the name, typeclasses aren't classes. They are more like interfaces.

In this case, the type is read: "given some type `a` that is an instance of `Show` , I will take an `a` and return a `String` .

Basically, the thing you call `show` on has to be "showable". Which most basic and even compound datatypes are.

Int, Float, tuples of showable things, and lists of showable things, are all showable.

Read and show (3)

The opposite of `show` is `read`.

`show` is a function that turns things into strings. `read` is a function that turns strings into other things.

Any type that supports construction from a string implements `Read`. For example, `Int` implements `Read`.

```
foo :: Int  
foo = read "123"
```

If you pass an invalid string, you just get a runtime error.

Mod, div, rem

Lastly, there's division. I put this off because it's annoying.

You can divide floats like this:

```
2 / 5 == 0.4
```

The `/` operator *only* works with floats, so here, `2` and `5` are being interpreted as floats (not casted, it's as if you wrote `2.0` and `5.0`).

You can't do this:

```
let x = 2 :: Int
    y = 5 :: Int
in x / y
```

Mod, div, rem (2)

The problem here is that the `/` function is only defined on Fractional numbers, and Int doesn't count:

```
(/) :: Fractional a => a -> a -> a
```

`Fractional` is another of those type class things. Floats and Doubles are fractional, and so are Ratios, but Ints are not.

So how do you divide integers?

Fixity

You use the `div` function: `div 5 2 == 2`

"Eww, do I really have to write it like a function?"

Sort of, but `div` is a binary function, meaning, it's a function of two arguments. Haskell lets you write binary functions in *infix* position.

The *fixity* of an operator tells us where it can appear. A prefix operator (which functions normally are) goes before its operand (argument).

A postfix operator goes *after*. `++` in C can be either prefix (`++x`) or postfix (`x++`) in that language. Haskell does not have any postfix operators.

Precedence

However, Haskell allows you to turn *any* binary function into an infix operator by putting backticks `` `` around it.

So you can do this: `5 `div` 2`, which returns `2`.

Normally, when we talk about operators, we also talk about precedence, which is the order of operators being applied. For example, `*` has higher precedence than `+`, because `1 + 2 * 3 + 4` is treated as `1 + (2 * 3) + 4`.

However, functions always have the highest possible precedence.

Mod, div, rem (3)

There are also `mod` for modulus, and `rem` for remainder.

Wait, there's a difference?

For positive numbers, no. But for negative numbers: `(-2) `mod` 5` actually is `3`, because the *mod* function treats numbers like they're on a clock.

Since `4` is the largest number we can have `mod 5`, `-1` is the same as `4 mod 5` (wrapping around from zero), and `-2 mod 5` is `3`.

If you haven't encountered this concept before, [here's an excerpt from a contemporary math textbook that explains it](#).

`rem`, on the other hand, is just the remainder. So `(-2) `rem` 5` is just `-2`.

But why did we need parentheses in `(-2) `mod` 5`? Well...

Precedence (2): A strange, unfortunate interaction

...Because functions (whether infix or prefix) have the highest precedence, that means unary minus signs do not.

So this: `-2 `mod` 5` means `-(2 `div` 5)`, and not `(-2) `mod` 5` like you would expect.

This is easy to forget, but the basic rule that causes it is easy to remember: functions have the highest precedence, no matter what.

Just remember, "*In functional programming, functions come first.*"

Programming language design

This is actually a good illustration of how programming languages can be "designed".

You might think it's dumb that `-x `mod` y` means `-(x `mod` y)`, and it kind of is, but how do you fix it?

There's a nice, simple rule in Haskell: functions first. Super easy to remember, and very useful when parsing expressions.

If you make it something like "unary operators first, then functions, then everything else", it's now more complicated. And what if you don't want unary operators to always come first? What if they need to be allowed to vary?

Programming language design (2)

Here, we are balancing simplicity versus intuition.

We could make things even simpler by removing operator precedence entirely, but then `x + y * z` would be `(x + y) * z`. That would fool a lot of people who are expecting a more mathematical order of operations.

But, unironically, for me, I think it might be worth it. I kind of like precedence-free languages, and I'll be showing you some later.

When you don't like something in a programming language, try considering what desires were being balanced, and what limitations prevented everything from being realized.

Questions?

Last Knowledge Check

1. Write a recursive function, fully typed, that computes the factorial. It should take and return `Integer`.
2. Write a point-free function and its type where it takes a string and prints three times its length.
3. Write a function with appropriate type that takes two leg lengths of a right triangle and returns the length of the hypotenuse.

Last Knowledge Check Answers

1.

```
fact :: Integer -> Integer
fact 0 = 1
fact n = n * fact (n - 1)
```

2.

```
thriceLength :: String -> Int
thriceLength = (3*) . length
```

3.

```
hypo :: Double -> Double -> Double
hypo a b = sqrt (a * a + b * b)
```

Next steps

Work through the chapter on [Simple input and output](#)

Work through the chapter on [Recursion](#).

The real grind: do some codewars! At this point, you should be able to do most Haskell problems of 8-kyu difficulty. Go [here](#) and start grinding!

This is the best practice. Don't forget to look at the top solution! It will sometimes be bizarre, but you can learn a lot from ultra-elegant Haskell code.